A Hypothesis on the Pattern of Life: OpenMorph, an Open-Source Library of Digital Organisms

Sébastien Mathieu
Morphic Intelligence Limited (Morphe.ai), New Zealand

Abstract

OpenMorph is an open-source Rust library for digital organisms. A **morph** is a small digital organism that grows parts when surprised, learns from every example right away, and lets go of what it does not use. We ask whether these few local rules produce properties used to mark the living. So far they do not set the morph apart: no life criterion we tested is unique to it. As a learner the morph matches classic baselines on some tasks and trails batch learners on smooth regression. Pinned runs replay bit for bit on every target we checked. On ARC-AGI-3 our live score rose from 0.22 to 0.29 out of 100 (the leaders: about 75), but the agent we submitted adds no units of its own: code we wrote earns its score. Our grown ARC-AGI-2 entry scores 0. Every number is sourced.

1. Introduction

In short: a morph builds itself from what it meets; this paper says exactly how far that goes today.

A morph lives on a **membrane**, a flat row of numbers called **cells**. The world writes some cells (the inputs) and teaches others (the targets). Small **units** read a few cells and each write one, so a change in the input ripples through the units until nothing changes any more. When the morph's output misses its target, it corrects itself on the spot, and if it stays surprised it grows a new unit. Every unit earns **energy** for being useful to the others, and a unit nobody relies on runs out and is removed.

The question. Is a small set of local rules enough to produce properties we use to mark living things: growing from experience, keeping itself going, losing what it does not use, depending on its parts? We do not claim a morph is alive. We propose one pattern precise enough to be wrong, build it as a library, and measure it against ordinary neural networks.

What we found. The rules are precise and well understood (§6.1), and every run is exactly reproducible (§6.4). But none of the life properties we tested is unique to the morph (§6.2), it is a modest learner (§6.3), and none of our ARC scores comes from the organism (§6.8). We report all of it, failures included.

Why this work. The author is autistic, and this project began as a personal question: why does my brain work differently, and, beneath that, how do a brain, life and reasoning emerge at all? OpenMorph is about ten years of trying to answer that question by building something and measuring it. It is given to the world for free, in the hope that others can do the best with it. The longer story is on the project's site, "Why OpenMorph".

Glossary. Each term is defined once here and used only in this sense.

term	meaning
cell	one number on the membrane; it has no type (a pixel, a sensor or a word are all the same to the core)
membrane	the flat row of all cells
unit	a small function that reads a few cells and writes one: `tanh(s·(Σw·x+b))`, with a gain `s` (1 unless changed)
readout	the unit that writes a target cell

term	meaning
feature	a unit the readout reads; grown features are the morph's hidden units
pool	all the grown features of one readout
event	one example: set the inputs, let the morph react, learn from the target
cascade	the chain of units that fire after an input changes; a unit fires only when a cell it reads changed
settle	the end of a cascade: nothing changes any more, or the event's firing budget (a cap on firings) is spent
surprise	a running average of the size of the readout's error
birth	a new feature, grown while surprise stays high
reliance	how much the readout leans on a feature: the size of its weight on it
energy	a feature's budget: it earns by reliance and pays a fixed upkeep each event
cull	the removal of a feature whose energy ran out (after a newborn grace period)
genome	the settings of the rules (`Config`); one default genome is used throughout
grown / learned / authored	made by the morph's own rules / a parameter fitted online inside code we wrote / written by us

2. Philosophy

In short: grow, do not design; keep it simple; measure, never claim.

OpenMorph is built on two rules.

1. **Nothing is authored.** Perception, features, gating and composition are grown and learned. The core never encodes a task: no hand-set weights, no per-domain driver, no hard-coded scores. If a mechanism must be written by hand to pass a test, it does not belong in the core.
2. **Simplest is most beautiful.** A real fix deletes machinery. A special case is a smell; the aim is a small, general mechanism from which the behaviour falls out.

The organism is what lasts. Interfaces, skills and tasks are disposable; a morph carries what it learned from one to the next.

Borrowed from living things. Each rule has a living model.

- *Cells and a membrane*: one flat space of values that every part reads and writes.
- *Growth where challenged*: a new unit is born where error stays high, as tissue grows under load.
- *Local learning*: each unit adjusts from what it sees, like a synapse, with no global teacher passing errors back through layers.
- *Metabolism*: units earn and spend energy, and what is not used is removed.
- *Nerves*: a unit reacts only to change, then rests.
- *Mycelium*: organisms share what they learned, device to device, signed and verifiable.

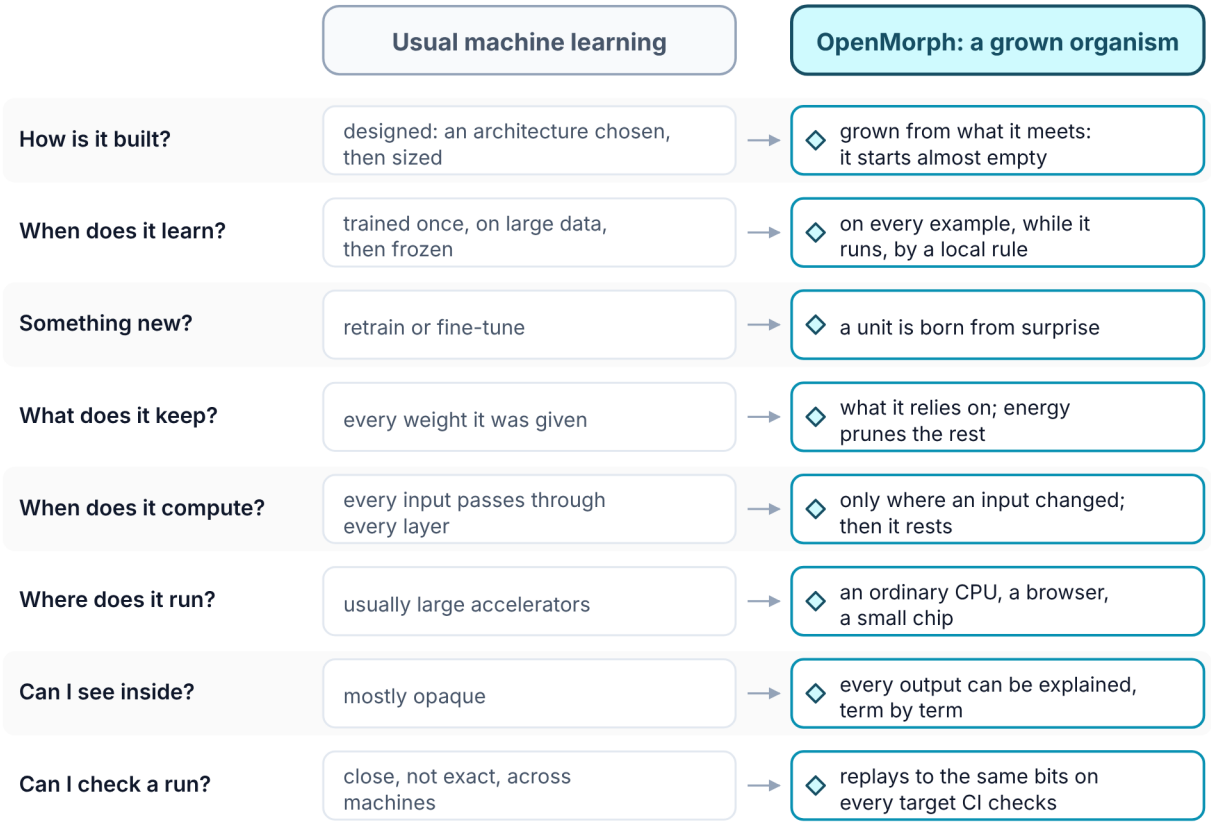
Truth, not trophy. We measure; we do not claim. Each claim carries a label: **proven** (a derivation whose test runs on the code), **measured** (a run, with its command, seeds and commit), **passed but shared** (a pre-registered bar met, but a simpler rival met it too), **conjecture**, **open** or **refuted** (missed its bar). Bars are written down before the run that tests them. Seeds are kept in separate blocks: one to develop on, one to confirm, and a **sealed** block read once. A claim that stops holding stays in the test suite, marked ignored, with its numbers. Every decrease is marked ▼, even within noise.[1]

[1] The labels and rules: `.claude/skills/measure/SKILL.md`, `docs/status/validation-registry.md`; an example pre-registration: `docs/research/1.9-streams/th-fresh-block-prereg.md`; refuted claims kept as ignored tests, for example `crates/openmorph-physics/tests/proofs/theory_fresh_claims.rs`. Seed blocks are registered in `docs/research/parameter-laws.md` §1 ("lab N-M" names a block).

3. The idea

In short: most machine learning is designed, trained once and frozen; a morph starts nearly empty and keeps growing and learning while it runs.

A typical network is designed by hand, trained once on large computers, then frozen and served. A morph starts with almost nothing. It grows the units it needs from what it sees, keeps learning while it runs, computes only where its inputs change, runs on an ordinary CPU or a small chip, can be looked inside, and replays exactly (Table 1, Figure 1).



A difference of design, not a claim of performance: the organism trains slower and costs more energy than classic learners at matched accuracy, and is cheaper only when inputs barely change (§5-§6). The right column is the default organism as the paper measures it; the left describes common practice in general terms.

Figure 1. Usual machine learning and OpenMorph, side by side, on the eight questions of Table 1. A difference of design, not a claim of performance (§6).

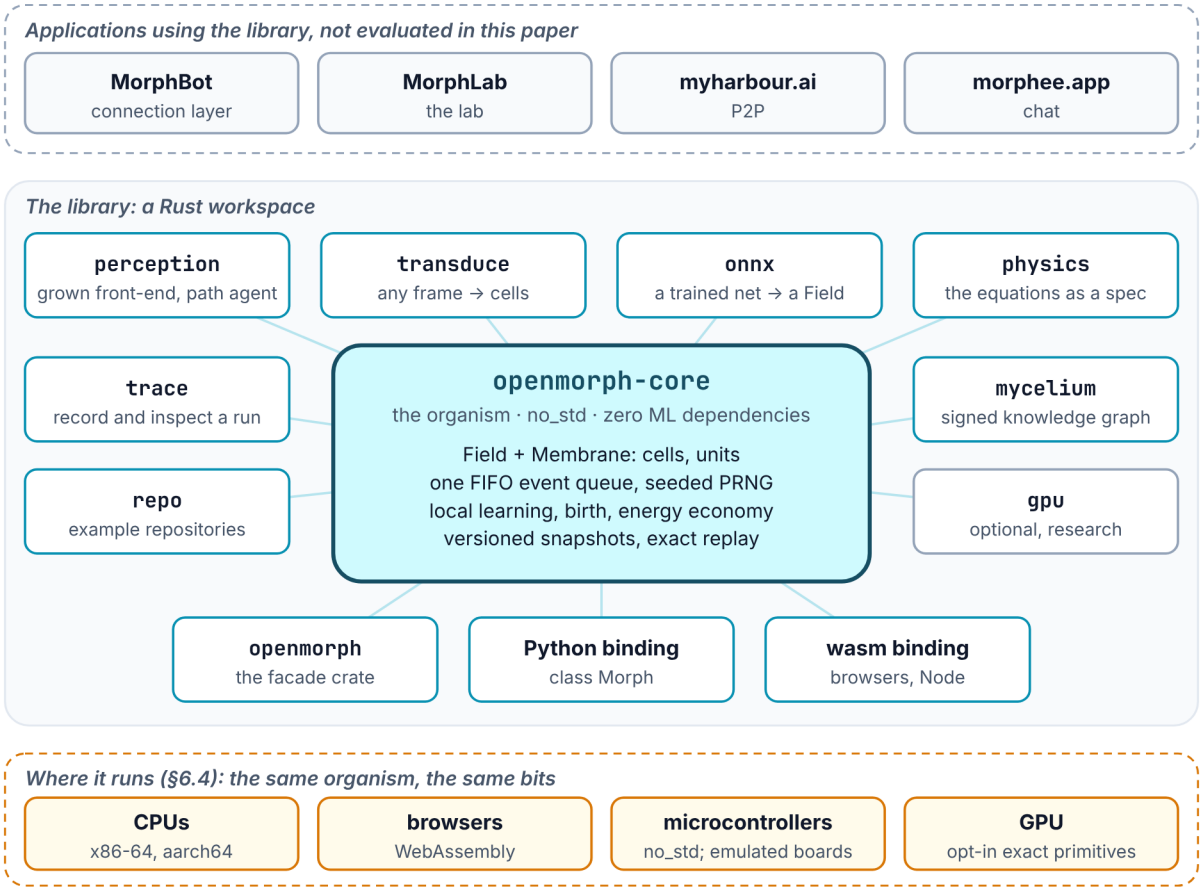
Table 1. What is different.

	usual machine learning	OpenMorph
how it is built	an architecture designed by hand	grown from what it meets: a unit is born where error stays high
when it learns	trained once, then frozen	on every example, while it runs
something new	retrain	a unit is born
what it keeps	every weight it was given	what it relies on; the energy economy prunes the rest
when it computes	every input, every layer	only where an input changed, then it rests
where it runs	large accelerators	an ordinary CPU, a browser, a small chip (§6.4)
can I see inside	mostly opaque	every output explained term by term (§6.5)
can I check it	runs differ slightly	a run replays to the same bits (§6.4)

The difference is in the design, not yet in the results: §6 shows where it pays and where it does not.

4. Architecture

In short: one small core holds the organism; everything else is a layer around it.



The core is the organism; the crates around it use it and do not change it. The applications pin a release and re-validate; their numbers are external and are not reproduced here.

Figure 2. The library: the organism at the centre (`openmorph-core`), the crates around it that use it without changing it, the applications built on it (not evaluated in this paper), and below, where the same organism runs to the same bits.

The library (Rust, with Python and WebAssembly bindings; Figure 2).[2]

- **openmorph-core** is the organism: the membrane and its units (`Field` in Rust, `Morph` in Python). It has no machine-learning dependency and runs without an operating system (`no_std`).
- **openmorph-physics** writes the organism's equations as a specification that can be edited and searched.
- **openmorph-perception** is a front end for grids and games, used by our ARC agents.
- **openmorph-transduce** turns any frame (text, image, sound) into cells, with no per-domain code.
- **openmorph-onnx** transcribes a trained network into a morph, cell for cell.
- **openmorph-trace** records and replays what happened inside a run.
- **openmorph-mycelium** stores knowledge as a signed, content-addressed graph that devices can share and verify.
- **openmorph-repo** reads, verifies and replays example repositories; **openmorph-gpu** (optional) runs batches of organisms on a GPU.

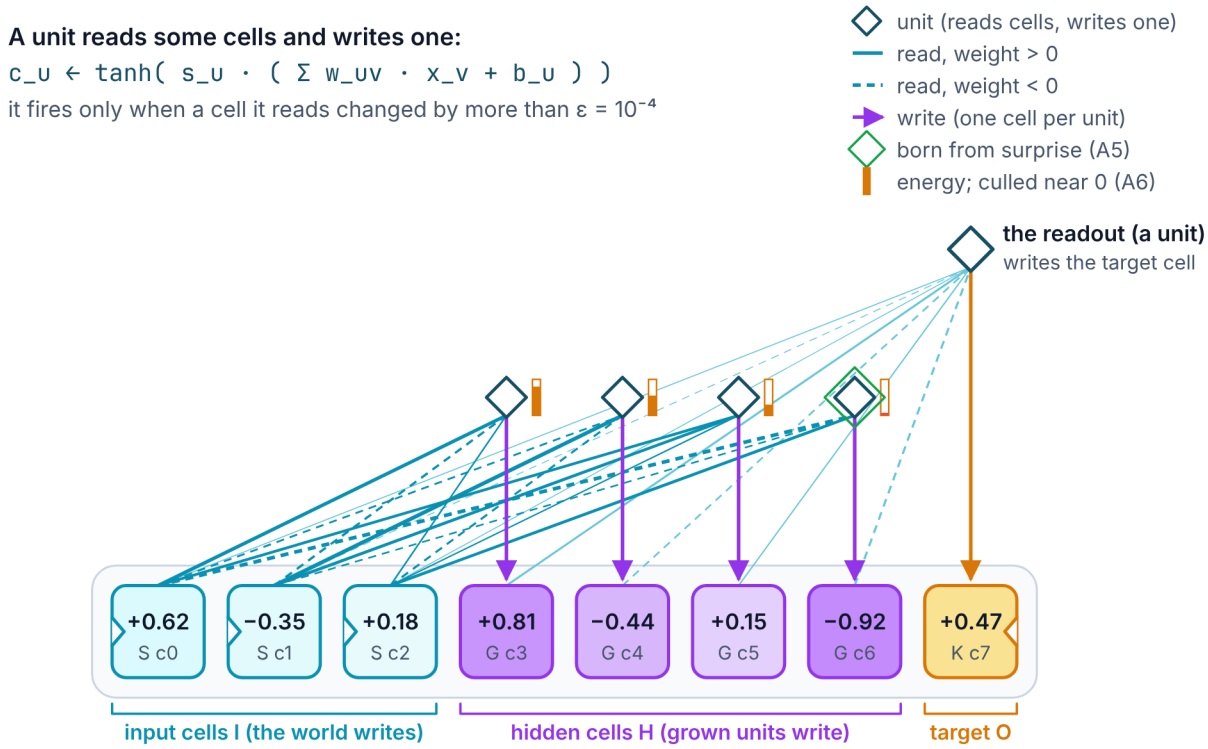
Around it sit applications that use the library: MorphBot (a connection layer), MorphLab (a lab), myharbour.ai (peer-to-peer sharing) and morphee.app (chat). They are named so the reader knows where the library goes; this

[2]Workspace members and licences: `Cargo.toml` and each crate's `Cargo.toml` at `19c01d16` (release v1.9.0). The core: `crates/openmorph-core/src/substrate/`.

paper evaluates none of them.

4.1 One event, step by step

In short: one event is a ripple, a correction, a possible birth, a payment and a possible removal.



The membrane: every cell, one flat vector of numbers (S input, G hidden, K target). A cell holds one value and never fires. The boundary I ∪ O is the caller's choice, not grown; a unit never writes an input cell. Values and weights drawn are illustrative.

Figure 3. Anatomy of a morph. Cells (squares) hold one number each and never fire; units (diamonds) read some cells with weights and write one, `tanh(s·(Σ w·x + b))`, only when a cell they read changed. Input cells are written by the world, the target cell is taught by it, hidden cells are written by grown units. Each feature carries an energy account (amber bar); a ring marks a feature just born. Which cells are inputs and targets is chosen by whoever sets up the morph, not grown. A schematic: values and weights are illustrative.

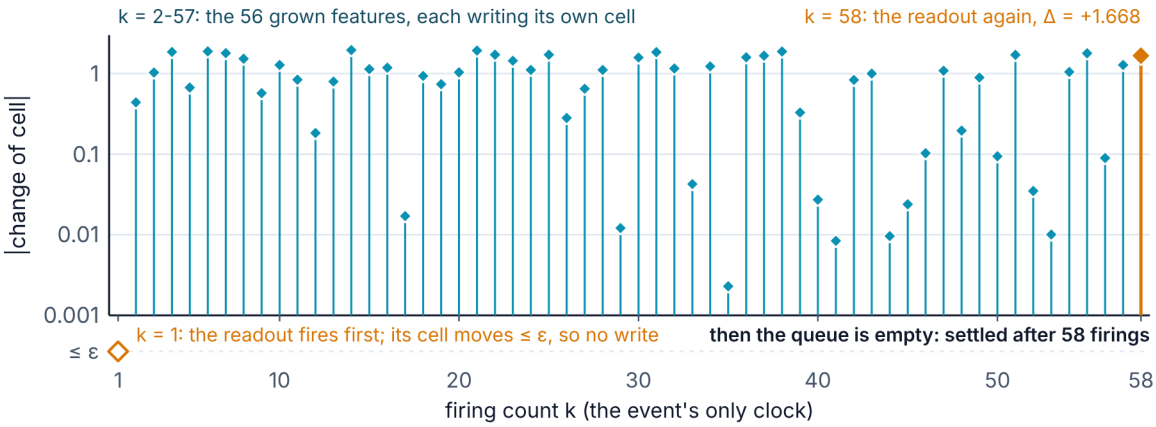
Figure 3 shows the parts; Figure 4 follows one recorded event through the steps below.

1. **Perturb.** The world writes new values into the input cells.
2. **Cascade.** Every unit that reads a changed cell joins one first-in, first-out queue. A unit taken from the queue writes its output only if the value moves by more than a small tolerance (10^{-4}), and that write queues the units that read the cell. Nothing fires at rest.
3. **Settle.** The cascade ends when the queue is empty or the firing budget is spent. There is no global clock; time is counted in firings.
4. **Learn.** The output is compared with the target. The readout takes one delta-rule step (its weights move against the error, with the step size capped). The features it reads get the same error signal one layer deep. Nothing further back is updated, and there is no backward pass.
5. **Birth.** The readout keeps a running average of its surprise, `S ← 0.9·S + 0.1·|error|`. While `S` stays above 0.1 and the readout has fewer than its maximum number of features, a new feature is born. Its weight in the readout starts at 0, so a birth never changes the output on its own (Figure 5).
6. **Meter.** Each feature earns energy in proportion to its reliance and pays a fixed upkeep: `e ← min(e + I·r/r̄ − U, cap)`, with income `I` = 2, upkeep `U` = 0.25, `cap` = 10, `r` its reliance and $\bar{r}$ the pool's mean. A

feature relied on less than an eighth of the average loses energy (Figure 6).

7. **Cull.** A feature whose energy falls to 0.1, after its newborn grace, is removed.

a **Settle: the world writes c0 ← +0.807 and c1 ← -0.002; their readers join one FIFO queue and fire in turn**



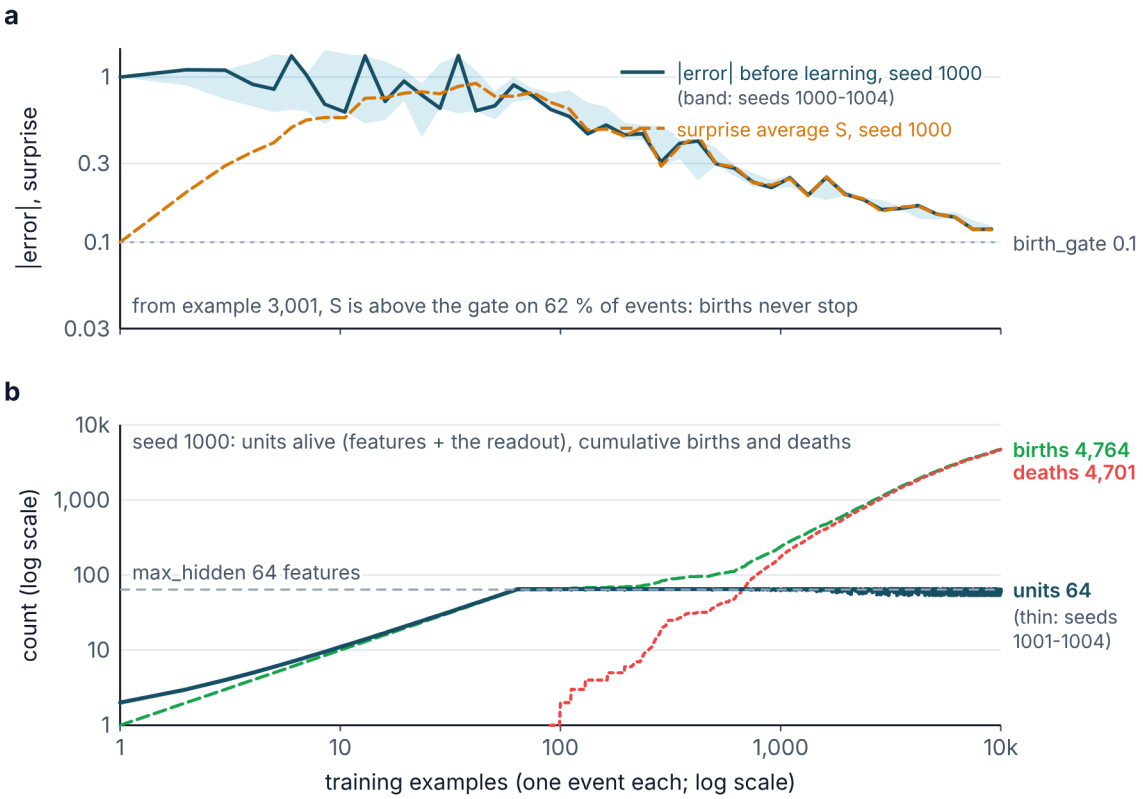
b **Then, in the same event (A3): learn, grow, meter, cull. Example 3,009 of the run**

error	target -1, prediction +0.669: |error| = 1.669	*the readout's cell after the settle, before learning*
learn	56 units move their weights: the readout and the 55 features it reads (largest step 0.111)	*a capped delta-rule step on the readout and one hop to the features it reads (A4)*
grow	surprise average 0.080 → 0.238 > birth_gate 0.1: one feature is born	*it reads 2 cells; the readout's weight on it starts at 0 (A5); energy 2.75*
meter	53 features stay at the cap of 10; 2 lose energy (up to -0.21)	*each is paid by how much the readout relies on it and pays an upkeep (A6)*
cull	one feature, at energy 0.28 before the event, falls under 0.1 and is removed	*units: 57 before, 57 after (one born, one culled)*

Recorded run: xor, the learning bench's seed 1000 (the paper's demo run), Config::default(); 57 units; ε = settle_eps = 0.0001. Firing order read by repeating the settle on clones at budget 1, 2, ...; data: docs/paper/data/figures/cascade.json.

Figure 4. One recorded event of the demo run (xor, seed 1000, example 3,009, 57 units). (a) The settle: the world writes the two input cells and their readers join one queue; the readout fires first but barely moves, so it writes nothing; the 56 features fire in queue order, each queuing the readout again; the readout fires last and moves the target cell by +1.668. The queue is then empty after 58 firings. (b) The rest of the event: the error is 1.669; the readout and the 55 features it reads move their weights; surprise rises from 0.080 to 0.238, above 0.1, and one feature is born; 53 features stay at the energy cap and 2 lose energy; one feature falls under 0.1 and is culled.[3]

[3]Figures drawn by `scripts/paper/fig_<n>.py` (style `scripts/paper/figlib.py`) from committed data or recorded runs (`docs/paper/data/figures/*.json`, written by the example `paper_figs`, which reads only the public `Field` API; organism code equal to release v1.9.0, `19c01d16`); captions and sources: `docs/paper/figures/README.md` (PR #1052). The xor runs reproduce every checkpoint of `docs/status/learning-bench/rust.tsv` for seeds 1000-1004. Figure 9's hashes are parsed from `ci/pins.env`, `docs/core/platforms.md`, `docs/platforms/microcontroller.md` and the GPU logs.



xor, Config::default(), lr 0.1, budget 1e6, the learning bench's seeds 1000-1004; seed 1000 is the paper's demo run: 64 units = the readout + 4,764 births − 4,701 deaths, test accuracy 0.96850. Late in the run S averages 0.137, above the gate, so the economy culls about as fast as surprise gives birth. Test accuracy, mean of 5 seeds: 92.5 % at 1,000 examples, 96.6 % at 10,000; every checkpoint equals docs/status/learning-bench/rust.tsv. Data: docs/paper/data/figures/growth.json.

Figure 5. Growth from surprise in the demo run (xor, seeds 1000-1004, 10,000 examples; seed 1000 is the run of §6.5). (a) The error on each example before it is learned, and the surprise average against the birth threshold 0.1. (b) Features alive, cumulative births and cumulative culls. On xor surprise stays above the threshold on 63 % of events after example 3,000, so births never stop: the economy keeps culling what the readout does not rely on. The run ends at 64 units, the readout plus 4,764 births minus 4,701 culls, with test accuracy 0.96850; mean of five seeds, 92.5 % at 1,000 examples and 96.6 % at 10,000.

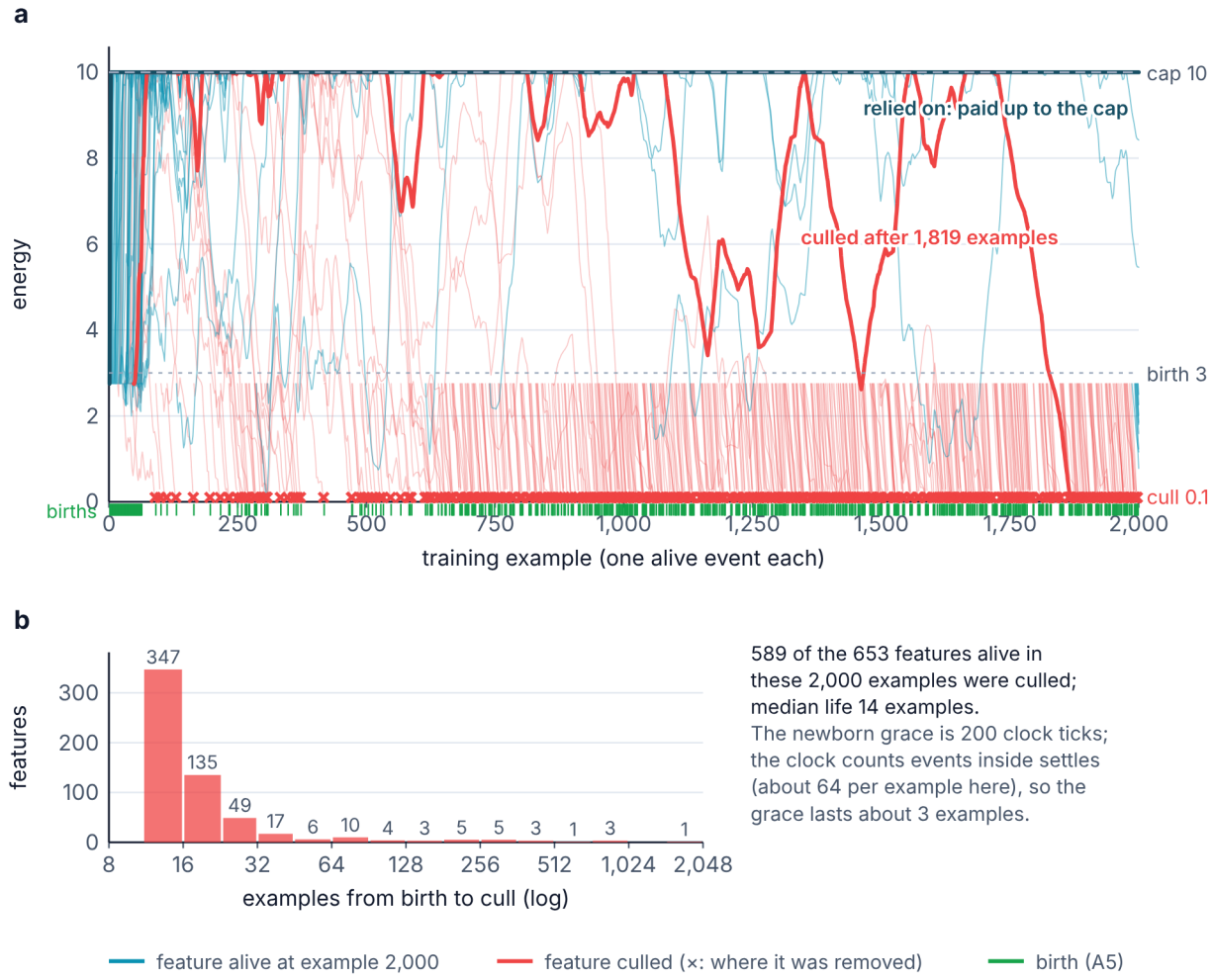


Figure 6. The energy economy at work in the same run (first 2,000 examples). (a) Every feature's energy: features the readout relies on climb to the cap (10); the others drain and are culled at 0.1 (×); green ticks are births. A feature earns more than it pays only while its reliance exceeds an eighth of the mean (`r/r̄ > U/I` = 0.125). (b) How long culled features lived: 589 of the 653 features of these 2,000 examples were culled, after a median life of 14 examples; the newborn grace lasts about 3 examples.

4.2 The rules, precisely

In short: seven local rules; this is the whole pattern under test.

Definition (morph). A morph is the tuple `M = (C, I, O, U, x, e, Θ)`:

- `C`, the cells, with values `x : C → f32`;
- `I ⊂ C`, the input cells, written only by the world, and `O ⊂ C`, the target cells, taught by the world;
- `U`, the units: unit `u` reads cells `R_u`, writes one cell `c_u`, holds weights `w_u`, a bias `b_u` and a gain `s_u`, and computes `F_u(x) = tanh(s_u·(Σ_{v∈R_u} w_uv·x_v + b_u))`;
- `e : U → [0, cap]`, each unit's energy, and Θ, the genome.

Traits. Each unit also carries a learning-rate factor and a plasticity. With the gain, these three traits can be copied, with variation, when a morph is reproduced (§6.2, C5). The internal cells reach the world only through the targets, and the world reaches them only through the inputs, so inputs and targets separate the inside from

the outside trivially, by how they are wired.

- **A1 (locality).** A unit reads only its cells and writes only its own cell.
- **A2 (fire on change, no clock).** A change above the tolerance queues the cell's readers on one queue; a unit is never in the queue twice.
- **A3 (one event).** In order: perturb, settle, learn, meter, cull.
- **A4 (local learning).** A capped delta-rule step on the readout, plus the same error signal one layer deep.
- **A5 (birth from surprise).** A feature is born while surprise exceeds 0.1 and the readout is below its maximum.
- **A6 (metabolism and death).** `e_i` $\leftarrow$ `min(e_i + I·r_i/`$\bar{r}$ `− U, cap)`; a feature at `e` $\leq$ `0.1` after its grace is removed, except a feature that has stopped learning (plasticity 0).
- **A7 (determinism).** One queue, one seeded random generator and one fixed maths library make a run a pure function of its inputs and seed. (Determinism is a property of the implementation, not a rule of life.)

These rules describe the default genome; opt-in settings change some of them and are out of scope.[4]

5. What is grown, what is authored

In short: on both ARC tracks the scores we report are not yet the grown organism's.

Every result says what the morph grew and what we wrote. **Grown** means the morph's own rules ran: features born from surprise, weights learned online by the local rule, energy metered and features culled. **Learned** (without grown) means a parameter fitted online inside code we wrote. **Authored** means written by us; **pretrained** means trained elsewhere before the run. Figure 7 and Table 2 apply these words to every result.

[4]The formal statement follows `docs/research/1.9-streams/th-life-criteria.md` §0 (merged in `ff9bec1`, PR #726), which indexes `docs/core/THEORY.md`. Code: `crates/openmorph-core/src/substrate/`; the defaults: `Config::default()`, `docs/core/config-reference.md` (tolerance `settle_eps` 10^{-4}, `birth_gate` 0.1, `max_hidden`, income 2, upkeep 0.25, cap 10). The traits are copied by `reproduce`.

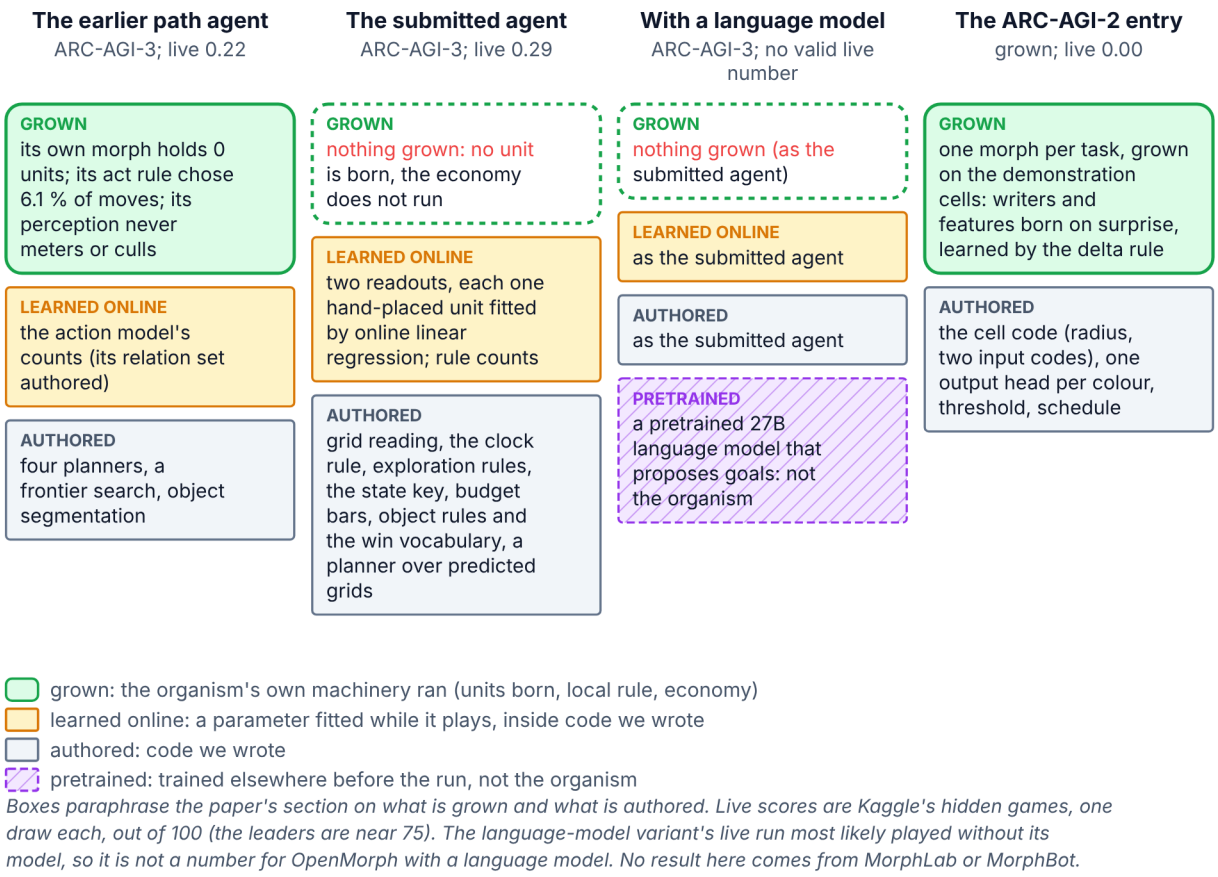


Figure 7. Grown, learned, authored and pretrained parts of our ARC entries (§6.8). The ARC-AGI-2 entry is grown. The submitted ARC-AGI-3 agent adds no units: its two OpenMorph parts are each one hand-placed readout, fitted online by recursive least squares (an online linear regression) on features we wrote. The language-model variant adds a pretrained model that proposes goals; it is not the organism.

Table 2. Grown and authored, result by result.

result	grown by the morph	authored by us (or pretrained)
learning, other kinds of data (§6.3, §6.7)	features born from surprise, weights learned online, culls; one default genome	the task's input and target cells; a generic symbol encoding for text and images; the genome's default settings (searched once, then frozen)
replay, portability, signing (§6.4, §6.5)	whatever the morph grew in the run	the test harnesses, the signing scheme, the GPU kernels
ARC-AGI-2 entry (§6.8)	one morph per task, grown on the demonstration cells	how a cell's neighbourhood is encoded, one output per colour, the threshold, the schedule; in the four-morph pool, the four variants and how answers are ranked
ARC-AGI-3, the earlier path agent (§6.8)	its own morph holds no units; its built-in move rule chose 6.1 % of moves	four hand-written planners, a search over untried moves, the object segmentation
ARC-AGI-3, the submitted agent (§6.8)	**nothing**: two hand-placed readouts fitted online, no unit born, no energy	how the screen is read, the exploration rules, the vocabulary of rules and win conditions, the planner
ARC-AGI-3 with a language model (§6.8)	as the submitted agent	as the submitted agent, plus a pretrained 27-billion-parameter language model
the life scorecard (§6.2)	the morph as shipped	the criteria and their bars, written down before the runs

6. Results

In short: the rules are well understood and every run is exact; the morph is a modest learner and not yet the source of any ARC score.

Table 3. Every result at a glance. Scores are on each benchmark's own scale.

result	headline	compared with	section
theory	28 proof tests pass; 4 statements proven, 2 passed (one of them shared), 2 refuted	—	§6.1
life criteria	none of those tested is unique to the morph; recovery failed, precariousness refuted	a growing and a fixed neural network	§6.2
learning	best or tied on 5 of 14 tasks; ▼ trails on smooth regression	12-13 classic learners	§6.3
replay	the same bits on every CPU, browser and emulated chip checked; on a GPU with an opt-in setting	—	§6.4
explain, sign, share	a full demo passes; explanations faithful on 2 of 4 tasks	—	§6.5
cost and energy	▼ training costs more at equal accuracy; inference cheaper only when inputs barely change	classic learners	§6.6
other data	text ▼ 1.2 points below logistic regression, inside the margin; ▼ images below it	n-gram models, logistic regression, kNN	§6.7
ARC-AGI-3 (live, out of 100)	0.22 → 0.29, coinciding with code we wrote; 94.5 % of the missing score is not finding the goal	the leaders, about 75	§6.8
ARC-AGI-2 (live)	0.00; 0 of 120 public evaluation tasks	a trivial baseline	§6.8

6.1 What the rules guarantee

In short: most of what is proven follows from the definition; four statements are new, and two predicted laws failed.

The statements are named by area: **T-S** for settling, **T-L** for learning, **T-E** for energy and **T-G** for growth. Each is derived in the project's theory notes and checked by a test that runs on the code; 28 such tests pass.[5]

Consequences of the definition (checks that the code does what the rules say, not discoveries):

- Flipping the sign of a feature's weights, its bias and the readout's weight on it changes nothing, because `tanh` is odd and reliance reads sizes (checked bit for bit, 4,000 events, 3 seeds).
- The default step is plain online gradient descent on the squared error, on the readout and its features only (T-L1). It does not adapt to the curvature of the error surface, which matters in §6.3.
- The pool's total income is fixed by its size, $N \cdot I$ per event, so energy is not conserved (T-E1).
- A feature gains energy only while its reliance exceeds an eighth of the mean (T-E2): the survival line of Figure 6.

New statements.

statement	what it says	status
T-S1	on a network without loops, a settle fires each unit a bounded number of times (one per distinct path length that reaches it); on 300 random networks the worst count reached the bound exactly	**proven**
T-S2	a settle that starts and ends within the tolerance stays within it, and which state it reaches inside that band depends on history (200 of 200 trials differ)	**proven**
T-S3	every firing order terminates when an entrywise bound on the network's sensitivity has spectral radius below 1 (the asynchronous contraction theorem of Chazan and Miranker, 1969, with a tolerance band); the condition holds on all 4,950 grown states tested (§6.2)	**proven**

[5] `cargo test -p openmorph-core --test proofs theory_` (`crates/openmorph-core/tests/proofs/theory_*.rs`; 28 tests at `ff80853`, 2.8 s); the table of statements and tests: `th-life-criteria.md` §0. Refuted claims: `cargo test -p openmorph-physics --test proofs theory_fresh -- --ignored`, read once on lab seeds 400-429 at `7e7a398` (`cargo run --release -p openmorph-physics --features lab --example th_fresh -- all --seeds 400-429`). T-S3's condition on grown states: C1a in `th-life-l1-l2-prereg.md` §6. T-L2 precisely: $e^{+}/e \in [1 - s \cdot \mu \cdot E \cdot f',\ 1)$, so $s \cdot \kappa < 2$ suffices (`docs/core/THEORY.md`). The non-terminating two-unit example: $\rho(D \cdot W) \leq 0.40$ while the entrywise bound's $\rho(M) = 1.8$.

statement	what it says	status
T-L2	one learning step shrinks the readout's error when the gain times the step size stays below 2	**proven**
T-E2, second part	which features survive a frozen readout: predicted right on 30 of 30 fresh seeds, but a simpler rule also was, and on 23 of 30 nothing died	**passed but shared**
T-G1	births become the usual state when the error's spread exceeds about 0.1253; measured 0.1133, inside the band [0.11, 0.14] written down before the run	**passed**

Refuted (written down before, read once on 30 fresh seeds):

- **T-L3**, a law for how fast the error shrinks, held on 12 of 16 tasks and failed on 4 (xor, pairs64, spirals, multi10), all where the step is near its stability limit.
- **T-L6**, a predicted forgetting lag of 1,000 events for one readout variant, measured 200, against 150 for a baseline that ignores the law.

A natural guess fails too: a spectral radius below 1 for the network's sensitivity does **not** make a settle end (a two-unit example never settles). A proposed energy law with a carrying capacity is a hypothesis, in Appendix B.

6.2 Is it life-like?

In short: no property tested is unique to the morph; recovery from damage failed and precariousness was refuted.

We listed nine criteria of life and turned each into a test with a bar written down first.[6] A criterion supports the hypothesis only if it is **distinctive**: the morph passes and networks that are not organisms fail. The same tests ran on two such networks: a growing and pruning online neural network, and a fixed one.

#	criterion	the morph	growing network	fixed network	reading
C1	keeps itself together	(a) its settling condition held on 4,950 of 4,950 grown states; (b) recovery after damage: 28 of 29 seeds on one task (needs 27) but not by regrowth (also 28 with births off), 18 of 29 on the other: **failed**	25 of 28 ▼ and 19 of 27: failed	22 of 27: failed	**no system recovers**
C2	metabolism	measured; a defect: one kind of grown feature never dies (64 born, 0 culled, 30 of 30 seeds)	has no energy	has no energy	**unique only by construction**
C3	growth from experience	measured (§6.3)	not run	not run	not compared
C4	adaptation to change	re-adapts; **refuted as an advantage**	not run	an online network of fixed shape re-adapts better on 3 of 3 held-out families	**not distinctive**
C5	reproduction and heredity	units: measured; whole organisms: **refuted** on the default genome (a copy differs from its parent by 0.386, against 0.0035 for an identical organism)	not run	not run	not compared
C6	agency	on toy tasks only; **refuted at task scale so far** (§6.8)	not run	not run	not compared
C7	precariousness (it collapses without input)	0 of 30 collapsed: **refuted**; also under a proposed conserving economy (0 and 1 of 25 against 23 needed, ▼ refuted)[7]	0 of 30	0 of 30	**nothing is precarious**
C8	its parts depend on each other	reliance predicts damage on 29 and 30 of 30 seeds	30 and 30 of 30	24 of 30	**shared**

[6] `docs/research/1.9-streams/th-life-criteria.md` (merged in `ff9bec1`, PR #726). Recovery (L1), precariousness (L2), C8 and C1a confirmed on lab 570-599 and 600-629 (`th-life-l1-l2-prereg.md` §6, merged in `0fb5965`, PR #753) and reproduced on `main` at `b5a9c5ac`; refuted claims kept in `life_scorecard_claims`. The two tasks for C1 and C7 are reg2d and lin-cond.

[7] The proposed conserving economy (Appendix B), frozen, on lab 1100-1124 (25 seeds, read once): the pool sits at 4 features, far below its capacity of 16, so a constant input removes none. `docs/paper/sub/metabolic-law.md` (PR #917, merged as `2939b445`); the seed block in `docs/research/parameter-laws.md` §1 (PR #928, `416f27f5`).

#	criterion	the morph	growing network	fixed network	reading
C9	open-ended evolution	not tested	—	—	not tested

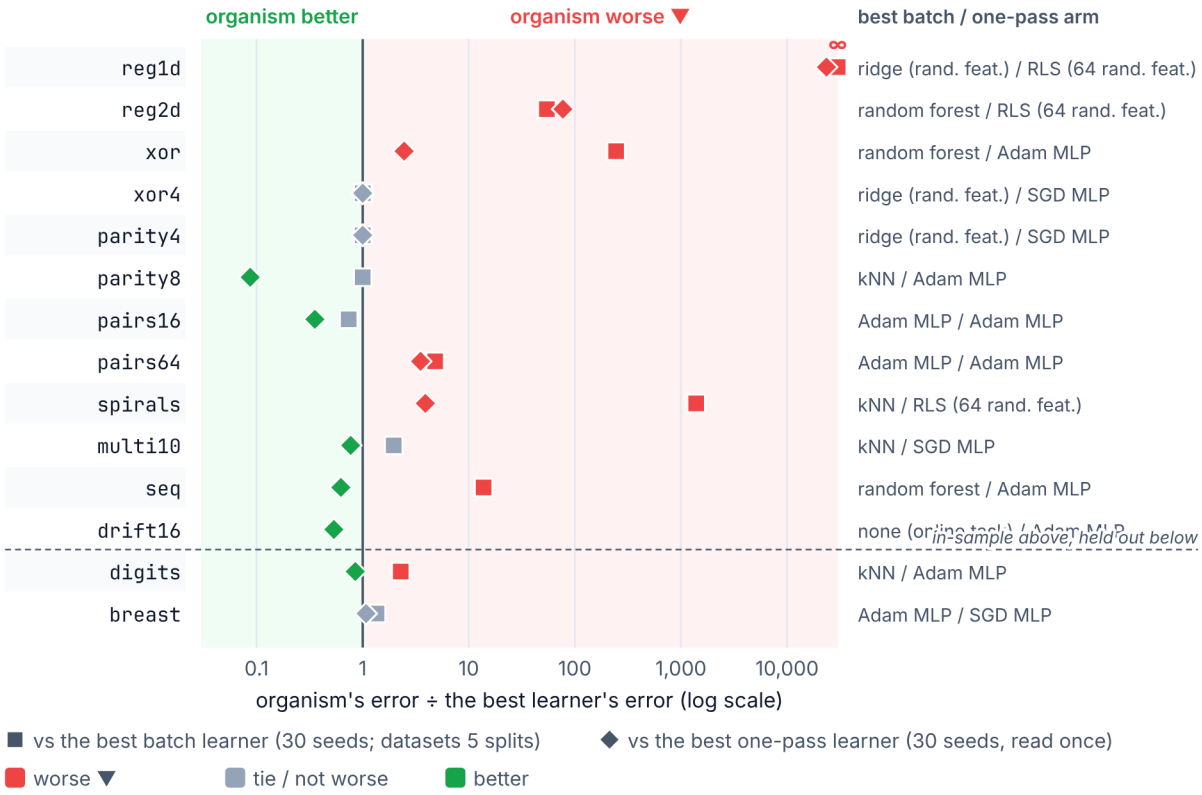
So two criteria failed outright (C1's recovery and C7's precariousness), two were refuted for the morph alone (C4's advantage, C5's whole-organism heredity), one is shared (C8), and one is unique only by construction (C2). We do not claim that a morph is alive, that it makes its own boundary, that it adapts better, or that it shows agency at task scale.

6.3 Learning against classic learners

In short: the morph keeps up on some tasks and trails batch learners on smooth regression; most tasks are ones its default settings were tuned on.

The default morph ran against 12 classic learners on regression and 13 on classification (least squares, ridge regression over random features, neural networks, random forests, k-nearest neighbours and others), at 30 independent seeds.[8] The 14 tasks are 12 synthetic ones on which the default settings were chosen (smooth regression in one and two dimensions, reg1d and reg2d; logic tasks xor, xor4, parity4 and parity8; feature-pair tasks pairs16 and pairs64; two spirals; a ten-class task, multi10; a sequence, seq; a target that changes halfway, drift16) and two real datasets it was not tuned on (handwritten digits and breast cancer). Figure 8 shows all 14.

[8] `docs/paper/sub/learning-parity.md` (`df475430`); `docs/core/learning-bench.md` and `learning-bench-independent.md`. The one-pass comparison: claim 11 of `learning-parity.md`, confirmation seeds lab 660-689 read once (`e9030f04`). Why reg2d trails: `docs/research/wall4-readout-plan.md` (development seeds). The opt-in setting: `Config::streaming()`, `CHANGELOG.md` [1.9.0] (`19c01d16`). Measured before the pin: the scikit-learn arms at `6992ebe5` (2026-10-15; digits and breast cancer at `578c233c`, 2026-10-14), the morph's rows at `ce26bf80` (physics epoch E1, 2026-10-16). Release 1.9.0 changed 82 of 1,420 bench rows slightly; a re-run at the pin (`learning_bench --summary`; `scripts/learning_bench/learning_bench.py all --independent`) is scheduled for 2026-11-07, and any number it moves will be updated here.



Error is NMSE on regression tasks and 1 − accuracy on classification tasks; a tie at 100 % accuracy plots at 1. Batch: the best of eight batch learners (least squares, two ridges over random features, gradient boosting, kNN, logistic regression, an Adam MLP, a random forest); worse or better means the best arm's mean beats the organism's by more than one of the organism's standard deviations (a descriptive band, not a test); drift16 has no batch arm; a best arm with zero error plots at the right edge. One-pass: the best of seven (online SGD and Adam MLPs, recursive least squares on raw inputs and on 64 random features, a Hoeffding tree), the report's verdict beyond 2 SE. breast = breast cancer. The 12 generated tasks are in-sample (the default genome was chosen on them); digits and breast cancer are held out. Config::default(), one online pass of 10,000 examples. Sources: docs/paper/sub/learning-parity.md (claims 1-4, 11), docs/status/learning-bench/, docs/status/param-laws/ro-online/conf/report.md.

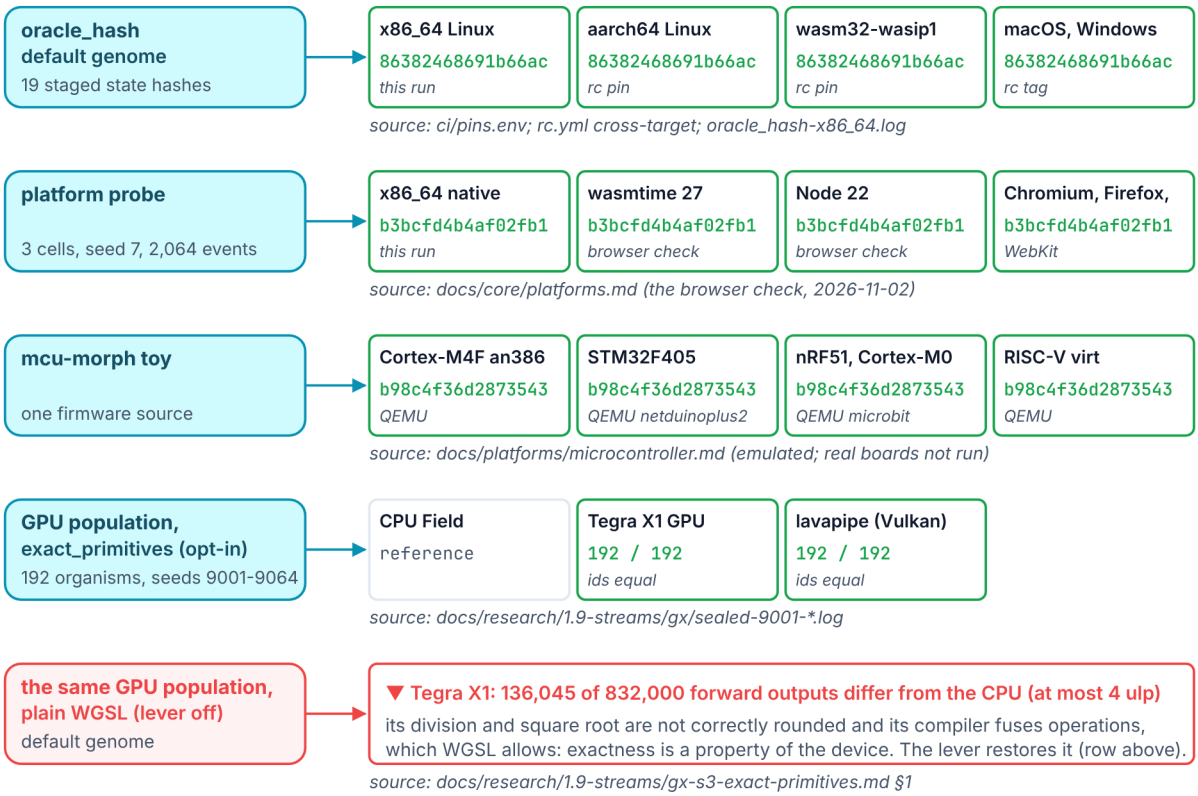
Figure 8. The morph's error divided by the best learner's on 14 tasks (log scale; left of 1 the morph is better): against the best of eight batch learners at 30 independent seeds (squares), and against the best of seven one-pass learners on a confirmation block read once (diamonds). ▼ marks where it trails beyond the descriptive band. The 12 synthetic tasks are in-sample; digits and breast cancer are held out.

- **Where it matches.** Its mean is the best or tied for the best on 5 of 14 tasks (xor4, parity4 and parity8, where many learners reach 100 %; pairs16; drift16 among online learners). On 7 of 14, including those, no learner beats it by more than one standard deviation.
- **Where it trails ▼.** On smooth two-dimensional regression (reg2d) it ranks 9th of 12, with error (NMSE) 0.0108 against 0.0002 for a random forest and 0.00063 for ridge regression over random features. Batch learners also lead on smooth one-dimensional regression (reg1d), xor, spirals, pairs64 and seq; one-pass learners lead on reg1d, reg2d, xor, pairs64 and spirals.
- **Held out.** Digits 96.7 % ▼ against 98.6 % for k-nearest neighbours; breast cancer 96.4 %, no learner better by more than one standard deviation.
- **Against learners that also see each example once** (a pre-registered comparison, read once). Against the best of seven one-pass learners, it is not worse on 9 of 14 tasks; the bar was 10 of 14, so it **fails**.
- **Why smooth regression trails.** Refitting ridge regression on the morph's own grown features reaches 0.0006, so the features are good enough and the online learning rule is the gap, as T-L1 predicts. Release 1.9.0 adds an opt-in setting that halves this error at about 1.65× the computing time ▼; the default is unchanged.

- **Speed ▼.** Per example it is 1.2× to 27× slower than a one-pass neural network in the same program.

6.4 Exact replay, everywhere we checked

In short: a run is a pure function of its inputs and seed, and gives the same bits on every target we tested.



A run is a pure function of its inputs and seed: one FIFO queue, one seeded PRNG, and every transcendental through the pinned libm crate (A7). Green: the hash equals the recording's on that machine. "this run": re-run for this figure on x86_64 Linux (rustc 1.99.1) and committed as a log; "rc pin": checked against ci/pins.env by rc.yml's cross-target job on every release candidate (macOS and Windows on an rc tag); browser check, QEMU, ids equal: the cited record. A hash certifies the final state, not the history. Not shown: physical microcontroller boards, or replay across a change of the libm pin or the snapshot format.

Figure 9. One recording, many machines, one fingerprint: the same final hash on x86-64 and ARM processors and WebAssembly; the same organism in three browser engines; the same toy run on four emulated microcontrollers; and, with an opt-in setting, the same 192 organisms on the CPU, on a Jetson GPU and on a software Vulkan driver; ▼ without that setting the Jetson's GPU differs. A hash certifies the final state, not the history.

- **Processors.** Tests pin a run's internal hashes, and continuous integration checks them on Linux x86-64, Linux ARM (aarch64) and WebAssembly on every release candidate, and on macOS and Windows on every release.[9]
- **Browsers and microcontrollers.** The same organism equals the native one in Chromium, Firefox, WebKit and Node; one learning event takes 2.1-3.4× the native 4.2 μs ▼. The same fingerprint is reached on four emulated microcontrollers (three ARM boards and one RISC-V); no physical board has been run.[10]

[9] `substrate_determinism` and `oracle_hash` (`ci/pins.env`); `.github/workflows/rc.yml`, `cross-target.yml`; `docs/core/determinism.md`.

[10] Browsers: `docs/core/platforms.md` (PR #903, merged as `8f96a7d9`; organism `b3bcfd4b4af02fb1`; Chromium 153, Firefox 155, WebKit 26.6, Node 22). Microcontrollers: `docs/platforms/microcontroller.md` (PR #904, merged as `53712d68`; QEMU `mps2-an386`, `netduinoplus2`, micro:bit nRF51, RISC-V `virt`; `DEMO-HASH b98c4f36d2873543`).

- **GPUs.** With an opt-in setting, each of 192 organisms (64 sealed seeds, three variants of the rules) ends with the **same fingerprint** on the CPU and on the GPU, on a Jetson Nano and on a software Vulkan driver (Mesa lavapipe), with 4.69 million values checked and none differing. The setting does division and square root in integer arithmetic and flushes tiny values to zero; it costs about 1.4× CPU time ▼. Without it, the software driver already matches the CPU (0 of 832,000 outputs and 0 of 895,600 learned values differ), but the Jetson does not ▼: 136,045 of 832,000 outputs (16 %) differed, by up to 4 units in the last place, and still 1.7-2.3 % after a first fix, because its division and square root are not correctly rounded. Only the batched forward pass and learning step run on the GPU; this is correctness, not speed.[11]
- *Not shown:* targets outside this list; replay across a change of the maths library or of the saved-file format.

6.5 Explain, sign, share and replay

In short: a trained morph can be explained term by term, signed, sent and replayed to the same fingerprint, and tampering is caught.

One program runs the whole story on a CPU in under 0.4 s, and every step passes the checks written before it ran:[12]

1. **Trained online.** Node A grows a morph on the xor task (seed 1000, 10,000 examples): test accuracy 0.96850, 64 features (4,764 born, 4,701 culled).
2. **Explained.** For the output, the program lists all 65 terms `x·w`, the bias and the `tanh`, which reproduce the output bit for bit, and ranks the features by reliance.
3. **Signed.** A signs a 795 B manifest that pins the 1.17 MB record of the run.
4. **Verified and replayed.** Node B checks the signature, regrows the morph using 8.8 % of its work budget, and gets the same fingerprint, the same explanation and the same accuracy.
5. **Tampering caught.** A changed manifest byte and a changed record byte are both refused.

[11] Software driver (Mesa lavapipe): `docs/research/1.9-streams/gx-gpu-design.md` (`4227473b`). The Jetson Nano's Tegra X1, the plain build: `docs/research/1.9-streams/gx-s3-exact-primitives.md` §1 (on `main` via PR #782); after a first fix and with the full fix: `docs/research/1.9-streams/gx/paper-paragraph.md`. Sealed seeds 9001-9064 × three genomes, run once per device: `cargo run --release -p openmorph-gpu --features backend-gpu --example xp_ids -- 64 200 9001`; logs `docs/research/1.9-streams/gx/sealed-9001-*.log`; the setting `Config::exact_primitives`, off by default (PR #782, merged as `46858ad6`).

[12] `bash scripts/demo/provenance.sh` (run at `c665382e`, merged in `a6fc18bf`, PR #730; pre-registered checks D1-D5, all pass; log `docs/paper/data/r18/demo.log`); also run on every push by `cargo test -p openmorph-mycelium --test provenance_demo`. Figure 10: `docs/paper/figures/fig_demo.py`.

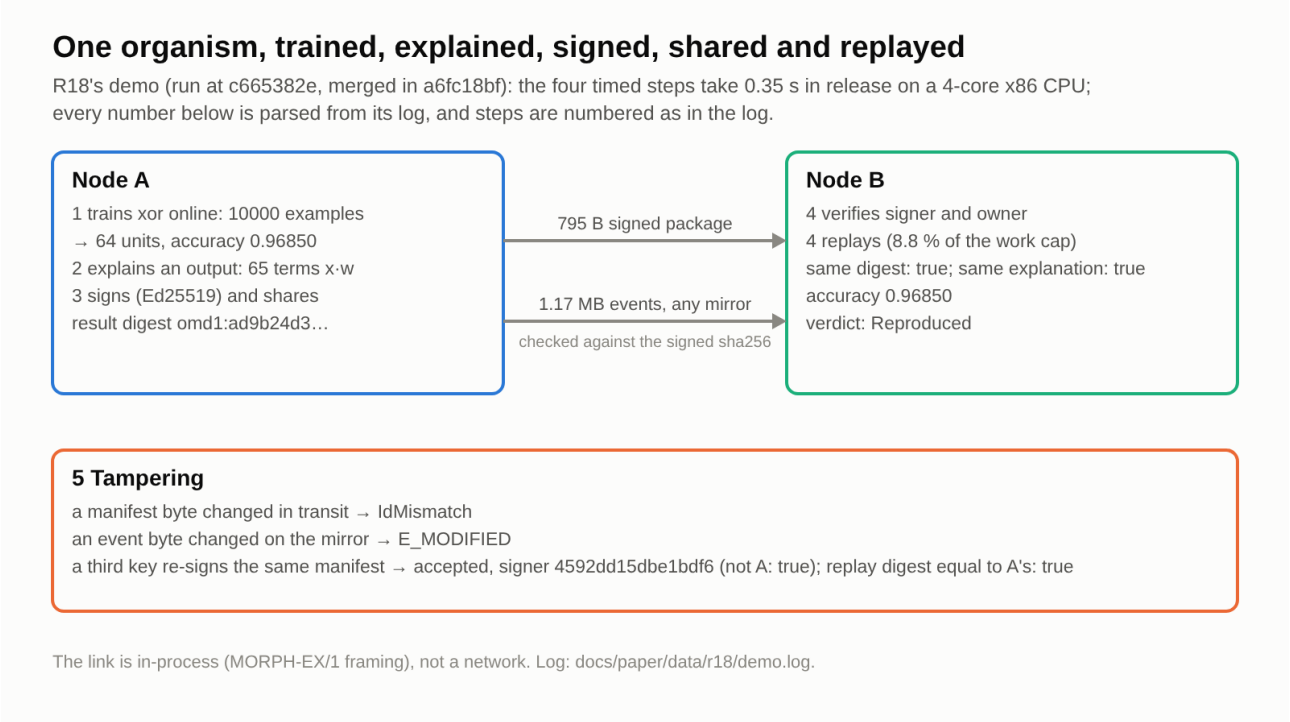


Figure 10. Node A trains, explains and signs; node B verifies and replays to the same fingerprint; tampering is refused. Every number is read from the run's log.

- **Are the explanations faithful?** Partly. Removing the features ranked highest by reliance does the most damage, with a median rank correlation of 0.51 against a bar of 0.5: a narrow pass, carried by two tasks (0.73 on reg2d, 0.58 on pairs16); on xor and spirals the ranking is near chance (0.14, 0.01).[13] This may help with auditing a model's decisions; it does not by itself make a model safe.
- **What a signature proves.** A two-node test passes 6 / 6 checks; the signed package is 2,368 B. A signature proves which key signed, not who did the work: a third party can sign the same manifest under its own key, by design. The fingerprint certifies the final state, not the path to it (a tiny change early in a run can wash out). The test runs in one process, not over a network, and the sharing layer has not been audited.[14]

6.6 Cost and energy

In short: a morph is small and runs on tiny hardware, but training costs more than classic learners, and inference is cheaper only when inputs barely change.

Costs are measured as CPU time and as counted machine instructions, not as joules.[15]

- **Small.** Each synthetic task trains and scores five seeds of 10,000 examples in 0.19-1.28 s on one core (multi10 in 11.8 s), in at most 8.1 MiB of memory; a grown morph holds 16-510 features. Measured as total wall time, that is ▼ 1.09× to 26.2× a small neural network in the same program (§6.3's 1.2× to 27× is per example). On an emulated microcontroller a morph trains for 2,000 events in a 15,176 B heap and a 2,680

[13] `docs/research/1.9-streams/r12-faithfulness-prereg.md` (30 fresh seeds, 101 qualifying pairs); the views `Field::explain`, `Field::forward`, `Field::unit_reliance` (`crates/openmorph-core/src/substrate/view.rs`).

[14] `cargo test --release -p openmorph-mycelium --test provenance_e2e` (PR #730, merged in `a6fc18bf`; blake3 ids, Ed25519 signatures). The washed-out change: one last-place change to one target, absorbed by event 3 (`7dfa92bf`).

[15] Small: `docs/paper/sub/small-compute.md` (`df475430`; the device demo `bash scripts/demo/mcu_provenance.sh`, PR #733, merged in `5196895e`). Training cost: `docs/research/energy/q1-results.md` (final seeds 950-969 read once; PR #891, merged as `a0ecd584`; reg1d 0.76×, pairs16 0.04×, xor4 3.1× to multi10 908×). Inference and continual learning: `docs/research/energy-q2q3-results.md` (seeds 5200-5229; PR #902, merged as `6e4e704f`; continual learning cheaper than retraining on 2 of 5 task families). Sharing: `docs/research/1.9-streams/en3-mycelium-distributed.md` §8 (sealed seeds 301-330; PR #910, merged as `8c7e37c8`). GPU speed: `docs/core/gpu-status.md` (PR #888, merged as `93d9c58a`; on the software driver the CPU stays 3-4× faster up to 16,384 organisms). Issues #885 and #886 track these studies.

B stack; signing it on the device as well needs ▼ 715,772 B of program memory and a 15,772 B stack, and a desktop then verifies it against its fingerprint (`5133ca5a...`). An emulated RISC-V chip needs 203,420 B of program memory and a 2,488 B stack.

- **Training costs more at equal accuracy ▼.** Against the cheapest classic learner that reaches the same accuracy, the morph is cheaper on 2 of 12 tasks and costlier on 7; on the other 3 it is the only learner to reach the target (once), never reaches it (xor), or no learner does (pairs64). Against a small neural network in the same program it costs 1.5-12.4× more.
- **Inference is cheaper only on nearly still inputs.** Because a unit fires only when an input it reads changes, a settled morph spends 691 instructions per step against 1,629 for a full pass when nothing changes. The two break even at about 0.3 % of inputs changing; when every input changes the morph spends ▼ 18,941.
- **Sharing.** Morphs trained on several devices merge exactly (to rounding, 5.4e-12), and a newcomer that starts from the merged result needs 5.5-8.7× fewer firings than one starting from scratch, or reaches what scratch never does. One round of distributed training still costs ▼ 1.4-4.9× a single central organism.
- **The GPU is slower everywhere we measured ▼**: on a Jetson Nano, 64 organisms × 200 rounds take 0.80 s on its CPU and 25.46 s on its GPU.

6.7 Other kinds of data

In short: a generic encoding carries the same morph to text and images, at a cost where values are ordered.

- **Text, images, sound.**[16] The same default morph is fed through a generic encoder that turns each symbol (a byte, a grey level) into its own cell, with no per-domain code.
  - *Text* (predict the next byte of English web text from the last 4 bytes, 10 fresh segments): 31.3 % correct, above a byte-pair model's 25.9 % and 1.2 points below logistic regression on the same cells (32.5 % ▼), inside the margin set beforehand. A five-byte model reaches 46.2 % ▼, and the morph grows about 2 million parameters ▼.
  - *Images* (8×8 handwritten digits, 30 fresh 70/30 splits): 94.10 %, below the same morph fed raw pixel values (96.94 %, ▼ −2.85 points) and logistic regression (96.44 %): it **fails** its bar. One cell per grey level loses the order of the 17 levels, and every learner pays for that (k-nearest neighbours ▼ −11.43 points). The raw-pixel 96.94 % and §6.3's 96.7 % are the same default morph under two harnesses: 30 random splits here, the learning bench's five stratified splits there.
  - *Sound* (eight generated tones): 100 %, tied with a standard frequency-analysis pipeline, so this test separates nothing.
- **A language model.** A morph trained as a byte-level language model is reported in Appendix C, apart from the results above, because it comes from an unmerged branch and has not been reproduced.

6.8 ARC-AGI-3 and ARC-AGI-2

In short: our ARC-AGI-3 score rose through code we wrote, not through the organism, and what is missing is finding a level's goal; our grown ARC-AGI-2 entry solves no evaluation task.

ARC-AGI-3 scores an agent out of 100, from how few actions it needs to complete each level of games it has never seen, against human players.

The live record (Kaggle's hidden games; one submission each; the submitted agent runs on the OpenMorph 1.8.0 core, §9).[17]

[16] `docs/research/1.9-streams/mo-modalities-prereg.md` §7.3, §7.6-§7.8 (PR #781, merged as `0bbb3e73`); the encoder `openmorph-transduce`. Text: FineWeb-Edu, lab 1030-1039; +5.41 points over the byte-pair model, 95 % interval [+3.90, +6.65]; ▼ −1.17 [−2.61, −0.03] against logistic regression. Images: lab 960-989; ▼ −2.85 [−3.16, −2.54]; ▼ −2.35 against logistic regression; adding each cell's scaled value recovers about half (95.59 %, still ▼ −1.33, lab 1050-1079). Sound: +85.70 points over a linear readout of the raw samples.

[17] `kaggle/arc-agi-3/proxy/README.md` and `docs/research/1.9-streams/dr-forensic.md` §8-§10 (branch `claude/1.9-dr-kit-x`, `4a5aa007`). Notebooks: v14 (0.31), v35, v38, v40 (0.22; the last is `main` at `8770cc65`), v43 (0.15), v46 (0.22), v48 (0.21), v51 (0.29; submission 57765246; branch `claude/1.9-ru-t2`, `2e7245ea`); every score checked against the owner's Kaggle account on 2026-11-06. The threading fix: `74920c61`, `2e7245ea`; the bug: dr-forensic §10 (`ad4ad884`, `a8d03a26`). Spearman ρ over nine agents: the proxy README.

agent	live score (out of 100)	what it is
an early agent (August)	0.31	one lucky game; the next twelve submissions scored 0.08-0.20
three releases of our path agent (planners and search)	0.22 each	the stable level of our agents through October
a rebuild of the early agent	▼ 0.15	—
two exploration agents	0.22, 0.21	exploration rules we wrote, around two hand-placed readouts
the submitted agent (29 October)	**0.29**	the last one, plus rules kept across levels and a threading fix

The leaders were at 75.56 (4,902 teams, 27 October). Our 0.29 is the first gain over the stable 0.22, not our best single score (0.31 was a fluke).

- **What coincided with the gain.** Two changes, which one live draw cannot separate or prove to be the cause: the first never ran live without the second, and our agents have spanned 0.08-0.31 live. The only replicate, a later agent (notebook v53, 0.29) most likely played the same path (see the language-model bullet below), so it repeats this result rather than testing it.
  (a) **Rules kept across levels.** The agent records what happens when the controlled object meets each kind of object (it moves, is blocked, pushes it, recolours it, removes it, dies), keeps those rules when a level ends, notes which conditions held each time a level was won, and plans toward them from level 2. We wrote the vocabulary; the counts are learned. On six offline game sets it went from 40 to 55 completed levels (sets on which the changes were developed), and, in a development build (`7cd921c4`), on seven unseen games from 10 to 12 levels (read once; its seed was not recorded). The submitted build also completes 12 there, with a lower mean score, ▼ 6.51 against 11.61.[18] (b) **A bug seen only live.** Kaggle runs all games as threads of one process, and two shared variables let one game send another game's click coordinates; the fix keeps them per thread.
- **Our offline tests did not predict live scores.** The submitted agent and the one before it score the same on 25 public games (0.7628), and over nine agents the public-game score and the live score are uncorrelated (Spearman $\rho \approx +0.02$).
- **What is missing: finding the goal.** Per level on 25 public games, against perfect play, 94.5 % of the score lost by an earlier version of the submitted agent comes from never discovering what completes a level, 4.7 % from misusing the controls, and 0.8 % from wasted actions (one seed).[19] A ladder of idealised agents on 29 offline games agrees: given a perfect model of the game but no goal it scores ▼ 10.79 (out of 100), the earlier path agent 39.58, and a perfect model with a full goal search 92.07 (harness `26b48ab4`; every rung written by us). A perfect model alone does not close the gap; only adding a full goal search does. Yet the oracle milestones of the last bullet below lost levels: they are relations that held along the solution path, necessary there but not sufficient elsewhere, so the missing goal is a state to reach, not a relation to satisfy. The first level alone is worth at most 3.52 points on average, so about 96 % of a game's score lies in later levels.
- **What carried the earlier path agent.** Its own built-in move rule chose 6.1 % of moves and completed 4 of 3,668 levels. Code we wrote carried it. On four small example games it scores 27.89 (random play 0.42). Switching off its four hand-written planners drops it to ▼ 1.15; switching off also its learned record of which moves undo or commute drops it to ▼ 0.23, below random. That record lets its search cover an 11×11 grid in 208 actions instead of 457. A change to the record raised one set of six unseen games from 41.25 to 58.77 (+17.52), almost all from two games; our in-repo benchmark missed it (its sealed score ▼ 45.4 → 45.3).[20]

[18] `docs/research/1.9-streams/ru-rules.md` (branch `claude/1.9-ru-rules`, `d5a94ab8`, PR #844); the six sets at seed 192 (the changes were developed on them); the seven unseen games (`env_think`) read once at the development build `7cd921c4` (seed not recorded), then on the submitted kit RU (`948fac3e`) at seeds 191 and 192, identical (the kit fixes its own seed): the same 12 levels, mean ▼ 11.61 → 6.51.

[19] `docs/research/1.9-streams/au-missing-element.md` (PR #832, `3378c2ad`; kit BH, `kaggle/arc-agi-3/kit-bh/` at `b0a9d49d`; 3,000 actions, seed 1). The ladder: `docs/research/1.9-streams/arc3-oracle-ladder.md` §2 (PR #737, `4627134e`; harness `oracle_ladder.py` at `26b48ab4`; its middle rung, L0, is the path agent with the M1 change, `3774565`; one seed; every rung written by us; on the 25 public games even the full-search rung reaches only 3.6-6.8). Level 1's cap: `lt-level-transfer-prereg.md` §1 (PR #812, `587202a9`).

[20] `docs/paper/data/arc/README.md` (at `3774565`); `docs/core/perception-path.md`. The paired change: `docs/research/1.9-streams/m1-prereg.md`, `kaggle/arc-agi-3/gate/results/m1/` (seeds 1-6, PR #701, merged in `7f933b5`; the two games `marsh` and `fray`). The built-in move rule's share (of 109,768 moves) and the mechanism audit:

- **Every route that gave the organism the decisions failed** (each pre-registered; the stage-0 read used development seeds). The morph's learned value ranked the state nearest the goal first on 0 of 10 levels it had failed; an agent driven by the morph's own value completed fewer levels than without it (−0.35 per run); teaching that value through moves imagined in a grown model of the game failed both its bars; a planner over that grown model lost ▼ −0.006 to −0.050 levels per run. Pre-training on 1,000 generated games did not help on new ones (18 → 14 levels ▼ on one held-out fold), carrying rules from one game to the next lost levels (55 → 48 ▼), and routing toward milestones given by an oracle lost levels too (36 → 21 ▼ on five held-out sets).[21]
- **With a language model** (a separate agent, never counted as the organism). Adding a pretrained 27-billion-parameter open model that proposes goals raised the offline count from 110 to 120 levels on the owner's GPU (a summary line; no per-set table is committed). The model's goal ideas reached 28 of 109 levels in one test, where a Claude model (version recorded in the data file) reached 83 of 102 in a similar one (one seed each). Its live run scored 0.29, like the submitted agent, but it is not known whether Kaggle gave that run a GPU, and it most likely played without the model; so 0.29 is not a language-model result.[22]

ARC-AGI-2 (scored by the share of test outputs right in two attempts).[23]

- Our entry grows one morph per task on the task's demonstration grids. It scores **▼ 0.00 live** and **▼ 0 of 120** on the public evaluation set (read once).
- In development, on training tasks it never saw, a pool of four grown morphs solves 4.68 % against 2.71 % for a single morph pair (406 tasks, a pre-registered pass). A trivial baseline (always answer the most common output) scores 0.67 % on a different split of 297 training tasks; it was not run on the 406.
- But 767 of the 1,000 training tasks are carried over from ARC-AGI-1. Every task our variants solved, but one, is one of those; on the 233 tasks new in ARC-AGI-2, nothing we tried solves any other. On generated tasks the morph solves 74 % of rules that look only at a cell's immediate neighbours, and 1 % of the others. A rule over single cells or single objects does not express ARC-AGI-2: relations between objects are the open problem.

7. Limits

In short: what does not work yet, plainly.

- **▼ No ARC score is the organism's.** The submitted ARC-AGI-3 agent adds no units of its own; the grown ARC-AGI-2 entry scores 0.
- **▼ None of the life criteria tested is unique to the morph.** Recovery from damage failed, precariousness is refuted, and the dependence of parts on each other is shared with a growing network.
- **▼ The learning rule trails batch learners** on smooth regression, and the one-pass comparison missed its bar. Most tasks are ones the defaults were tuned on.
- **▼ Training costs more** than classic learners at equal accuracy, and the GPU is slower than the CPU everywhere we measured.
- **▼ The energy economy has gaps.** A newborn's starting energy is created rather than drawn from any account (in one study, about 22 times the energy from all the stated sources); one kind of grown feature never dies; and there is no limit on the pool's size besides a hard maximum.

`docs/research/1.9-streams/mechanism-audit/README.md` (PR #743).

[21] The value ranking: `gs-prereg.md` (the goal screen); the morph-driven agent: `th-agent-stage0-prereg.md` (result `da4bc710`, development seeds 250-279); imagined training: `th-agent-1a-prereg.md` (PR #722); the planner: `pl-prereg.md` (PR #780, `ab1ce76b`, 180 paired runs); pre-training: `bt-train.md` §3, read 1, fold F2, seed 211, and the verdict §6 (PR #831, `b39a4094`); carried rules: `bt-train.md` §9 (branch `claude/1.9-bt-goal`, `29478728`, seed 191); oracle goals: `lg-llm-goal.md` (PR #841, `c7f04b42`); level transfer, 129 → 126 levels ▼: `lt-level-transfer-prereg.md` (PR #812); the oracle milestones, 36 → 21 ▼: `bt-train.md` §8, read M1 (branch `claude/1.9-bt-goal`, `29478728`; seed 191; kit DG13 with and without them), and why relation goals fall short, `lg-llm-goal.md` §4.4. All under `docs/research/1.9-streams/`.

[22] `docs/research/1.9-streams/ol-plan.md` (PR #859, `788789c1`; tracker issue #848); `kaggle/arc-agi-3/kit-ol/AGENT.md` lines 117-118 (branch `claude/1.9-ol-w6`, `eddcc953`; a summary line, no per-set table committed); the Claude reference, with the model's version: `ol-w5-goal.md` §3.1 (branch `claude/1.9-ol-w5`, `2dbd1e5c`); the live run (v53, submission 57819755): `kit-ol/README.md` (branch `claude/1.9-ol-w8`, `bd688dc8`).

[23] `docs/paper/sub/arc-agi-2.md` (PR #839, `59b976dd`), whose claims table names each file, command and commit; live submission 57548013; the single pair's earlier confirm read, 2.02 % against the 0.67 % baseline on 297 tasks, was not shown above it (95 % interval [−0.34, +3.37]); the carried-over tasks: `a2t-stages-prereg.md` §1 (PR #814, `e5117257`); the generated tasks: `tx3-prereg.md` §9.2 (PR #760, `012ab582`). An unverified audit adds that 32.5 % of evaluation tasks change the grid's size, which the entry cannot do.

- ▼ **Explanations are faithful on only two of four tasks**, with no user study.
- ▼ **Hardware is emulated.** No physical microcontroller has been run; sharing has run in one process, not over a network, and has not been audited.

8. Related work and theories

In short: a morph borrows from growing networks and artificial life, and shares parts of several theories of life while lacking the core of each.

Prior work. ARC measures skill-acquisition efficiency on tasks unseen in advance (Chollet 2019); ARC-AGI-2 raised the bar on composition and context (Chollet et al. 2025), and ARC-AGI-3 moves it to interactive games scored against human players (ARC Prize Foundation 2026). Growing a network as error demands goes back to cascade-correlation, which adds a hidden unit when learning stalls (Fahlman and Lebiere 1990), and to growing neural gas, which inserts units where error accumulates (Fritzke 1995); a morph grows from surprise in the same spirit, but asynchronously, and it also culls by energy. Digital organisms that compete for processor time and memory and evolve go back to Tierra (Ray 1991) and Avida (Ofria and Wilke 2004); a morph learns within a lifetime rather than evolving across generations. Growing neural cellular automata learn local rules that build and repair a pattern (Mordvintsev et al. 2020), but by backpropagation offline, where a morph learns online and locally. The readouts of our ARC-AGI-3 agent are fitted by recursive least squares, a standard online linear regression (Haykin 2002).

Theories of life.

- **Self-reproducing automata** (von Neumann 1966). A morph copies units and organisms but has no constructor that builds itself from parts, and whole-organism heredity is refuted on the default genome.
- **Closure to efficient causation** (Rosen 1991). A morph's repairs (births, culls) come from its own rules, but the rules are fixed from outside, so it is not closed in Rosen's sense.
- **Autopoiesis** (Maturana and Varela 1980). A morph makes and removes its parts; its boundary is set by hand.
- **Autocatalytic sets** (Kauffman 1993). The survival line is a condition for a feature to be sustained by the others' reliance; it is not a model of catalysis.
- **The free-energy principle** (Friston 2010, 2013). A morph has an inside and an outside by construction and grows from surprise, but it learns by a delta rule, not by variational inference, and our morph shows no agency at task scale.
- **Assembly theory** (Sharma et al. 2023). We have not measured an assembly index for grown structures; it could test open-ended evolution (C9).

The references were checked against their DOI, publisher or a library record.[24]

9. Why it matters, and how to help

In short: a small, exact, inspectable learner is useful before it is life-like, and every gap above is a concrete task.

What holds today is a learner that builds its own structure online, explains each output term by term, gives the same bits on a CPU, a browser and, with an opt-in setting, a GPU, and can be signed and shared. What does not hold yet is the strong form of the hypothesis. The most useful open problems (all of them, with code entry points and how a fix will be judged, are in Appendix A):

1. **A goal the organism finds by itself**: 94.5 % of our ARC-AGI-3 gap.
2. **Rules over relations between objects**: what ARC-AGI-2's new tasks need.
3. **Learning beyond one layer**: the gap to ridge regression, and rare exceptions such as walls.
4. **An economy that conserves energy**: births paid from an account, and a natural limit on size (Appendix B).
5. **Physical hardware and a real network.**

Each fix is judged the same way: write the bar down first, develop on one block of seeds, read a sealed block once, and keep the result whether it passes or not.

[24] `docs/paper/latex/refs.bib`; what remains unchecked: `docs/paper/latex/UNVERIFIED.md`.

Code and licence. The submitted ARC-AGI-3 agent's code is MIT-0: the agent, `openmorph-core` and the Python binding's `Morph` part; the perception crate stays LCL-1.0. The wheel actually submitted (sha256 `45bfde8b...`) also contains compiled perception code that the agent never calls. A wheel built from the public MIT-0 tree alone (the OpenMorph 1.8.0 core and the binding's `Morph` part, no perception; sha256 `5367a9f1...`) replays the submitted entry identically on all 58 gate rows (110 levels; mean of set means 25.2733). The entry runs on the 1.8.0 core, not on release 1.9.0, whose results the rest of this paper reports: on the 1.9.0 core the same agent completes ▼ 100 of those 110 levels (mean 22.8999).[25] Patent pending (NZ provisional, Petitgen Limited).

Appendix A. Open problems

#	problem	what is known	where to start	how a fix is judged
P1	**The organism deciding at task scale.**	Every route that gave the morph the decisions on ARC-AGI-3-style games failed or lost levels (§6.8).	`crates/openmorph-perception/src/agent.rs`	a bar on the offline real-engine games, on sets the change was not developed on
P2	**Heredity of whole organisms.**	A copy differs from its parent by 0.386 on the default genome (refuted); it holds on the older genome.	`crates/openmorph-core/src/substrate/evolve.rs`	a trait that moves under selection through energy, beyond a no-selection control
P3	**Smooth regression.**	NMSE 0.0108 ▼ against 0.00063 for ridge; the online rule is the gap (T-L1); an opt-in setting halves it at ▼ 1.65× the time.	`crates/openmorph-core/src/substrate/learn.rs`	the learning bench at 30 seeds, no task worse
P4	**Faithful explanations on every task.**	A narrow pass (median 0.51) from two of four tasks.	`crates/openmorph-core/src/substrate/view.rs`	the same removal test on fresh seeds, per task
P5	**Sharing over a real network.**	6 / 6 checks in one process; no network; no audit.	`crates/openmorph-mycelium/src/`	the same test over a network, and an external review
P6	**Physical devices.**	Emulated microcontrollers only.	`examples/mcu-organism/`	timing and energy on a physical board, with the same fingerprint
P7	**The refuted laws.**	T-L3 and T-L6 failed; two further statements are conjectures.	`docs/core/THEORY.md`	a corrected law, read once on fresh seeds
P8	**Open-ended evolution.**	Not tested.	—	a design for the test is itself a contribution
P9	**Staying alive by its own actions.**	In a world where food must be found by acting, no learning morph outlived a random forager for long; a hand-written forager lived about 24× longer. With a proposed conserving economy the pool's size tracks its capacity, but the morph still lives ▼ 0.69-0.87 as long as random play; the cause is how its learning target is scaled, and no variant tested both grows and lives on most seeds.[26]	`docs/research/1.9-streams/ro-survival-design.md`	a design that passes development, then a sealed block read once, against random play and both networks
P10	**Settings set by the organism's own state.**	A search for such laws found none better than a fixed constant or a clock (30 fresh seeds).	`docs/research/1.9-streams/s4-reflection.md`	a law that beats both on fresh seeds
P11	**Finding the goal, and transfer across levels.**	94.5 % of the ARC-AGI-3 gap is the goal; about 96 % of a game's score lies past level 1; carried rules, oracle goals and level transfer lost levels (§6.8).	`docs/research/1.9-streams/au-missing-element.md`	a goal the organism finds, on unseen or hidden games, with the bar written first

[25] `docs/release/arc-entry-replay.json` (PR #1063, merged as `4ac51860`), runs A (the submitted wheel `45bfde8b...`), G (the committed MIT-0 wheel, `kaggle/arc-agi-3/kit-ru/wheel/`, sha256 `5367a9f1bfab1ef8cb36e59e36c12d6c33992dbf3fec912643a31234c4eec367`) and D-E (the 1.9.0 core); six gate sets, seeds 192-193, 1,000 actions; per-row digests; the method: `docs/release/arc-entry-licence.md` §5, §7.

[26] Development seeds lab 10-39 and 720-749; sealed lab 690-719 read once (`alive-levers/a8a9-sealed-690-719.md`, PR #762, merged as `99dc99a1`): the energy budget balances, the pool tracks its capacity on 27-30 of 30 seeds, the morph lives ▼ 0.69-0.87 of random; a learned reserve survives famines (30 of 30 against 28 of 30 deaths, conditional on rescue at zero units) while three predictions fail ▼, one with an untested explanation (`th-metabolic-law.md` §8.14, PR #927, `7fa88fc0`). The cause (lab 1100-1124, read once; `alive-levers/a9-horizon-1100-1124.md`, PR #920, merged as `1858716a`): a fixed scale lives about 2.4× longer on 24 of 25 seeds but grows on 1; a 20-step horizon passes narrowly (3 of 25 grow; median ▼ 0.722 of random). `docs/paper/sub/metabolic-size.md`; `alive-prereg.md` (branch `claude/1.9-readout-laws`, `ecd8d84e`, `b5214460`): births create about 22× the stated energy (`wiring.rs:48`).

#	problem	what is known	where to start	how a fix is judged
P12	**What ARC-AGI-2 adds.**	Nothing we tried solves the 233 new tasks but one.	`docs/paper/sub/arc-agi-2.md`	units that are objects and rules that relate pairs of them, read once on unread new tasks
P13	**Learning rare exceptions; sharing a law across places.**	A diagnosis, not yet measured: credit spreads evenly over active inputs, so a rare exception (a wall) is not learned, and what is learned in one place is not reused in another.	`crates/openmorph-core/src/substrate/learn.rs`	a toy with rare exceptions and a held-out place, then the real games

Appendix B. A proposed energy law (a hypothesis)

In short: a conserving energy law would give the morph a natural size limit; it is proposed, partly tested and not shipped.

The proposal shares each feature's energy by its reliance: `dE_u/dt = b·s_u + α·LP_u + κ·intake_u − c`, where `b` is a steady income for being alive (**breath**), `s_u = r_u/r̄` its share of reliance, `α·LP_u` is paid when its errors fall (**progress income**), `intake_u` is food from acting, and `c` is the cost of a firing. With `b > c`, breath alone keeps the pool alive; with `b < c` the pool would starve without new things to learn, which would make it precarious; that prediction was tested and refuted (§6.2, C7). Summed over the organism, the law keeps an **energy budget** and implies a **carrying capacity**, `N* = (b + κI + α·LP)/c̄` ($\bar{c}$ the mean cost), if the shares sum to 1. The shipped economy has no such limit, because its income grows with the pool. In this form a morph would be an open, driven system that keeps its order by taking in energy, in the sense of Prigogine and Schrödinger (Nicolis and Prigogine 1977; Prigogine 1978; Schrödinger 1944); that is a framing, not a result.[27]

Appendix C. Unreproduced: a language-model morph

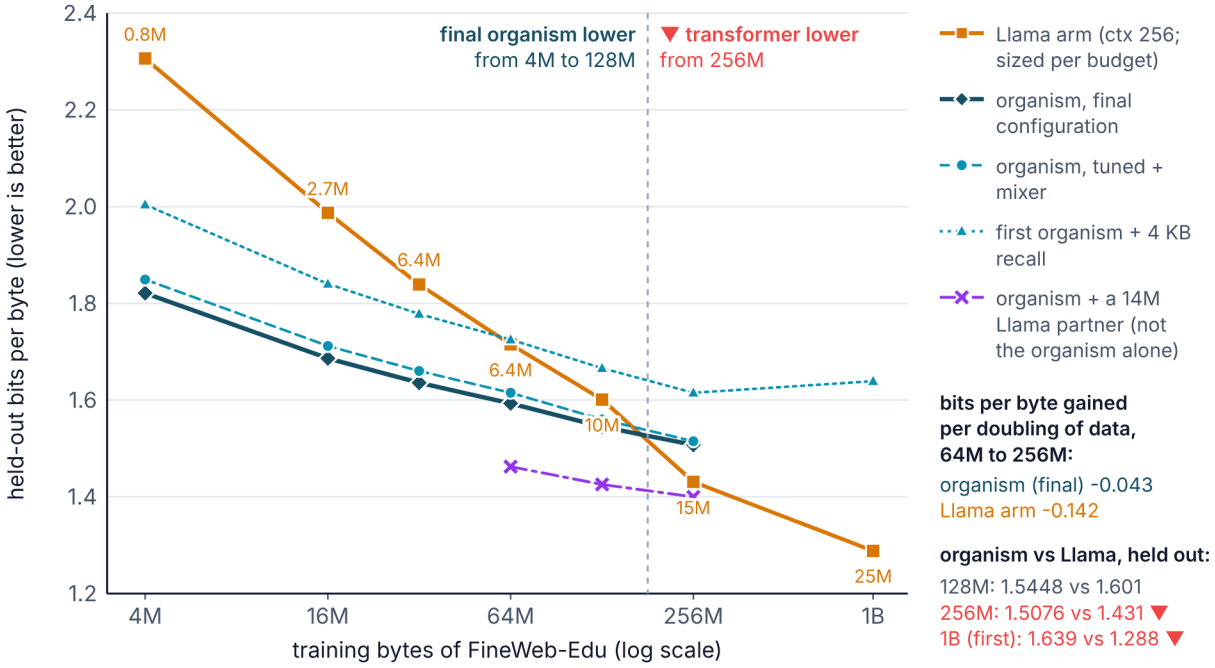
These results come from an unmerged research branch. Nothing here was re-run for this paper, and the public code does not reproduce them.[28]

A morph trained from scratch to predict the next byte of English web text (FineWeb-Edu; held-out bits per byte, lower is better, nothing learned while scoring) was compared with Llama-style transformers trained on the same bytes and sized for each budget: 10 million parameters at 128 million bytes, 15 million at 256 million, 25 million at 1 billion. Same bytes does not mean the same passes: the transformer trained several passes, the morph one.

- **Small data.** The morph's final configuration is lower up to 128 million bytes (1.5448 against 1.601).
- **Scaling ▼.** Each doubling of data from 64 to 256 million bytes gains the morph about a third of what it gains the transformer (−0.043 against −0.142 bits per byte). From 256 million bytes the transformer wins (▼ 1.5076 against 1.431); at 1 billion bytes an earlier morph scores ▼ 1.639 against 1.288.
- **Not the morph alone.** Figure 11's lowest line mixes in a small transformer partner.

[27] `docs/paper/sub/metabolic-law.md` (PR #773, merged as `da30d60b`; names in PR #779, `076a0ccf`; PR #786, `dedb9859`); `docs/research/1.9-streams/th-metabolic-law.md` §5-§6.6.

[28] `docs/research/morph-lm-summary.md` §2-§3, which quotes the branch's documents and re-ran nothing; the code is on the unmerged branch `claude/intelligent-shannon-b6n6pj` (`1755a3f6`); the gains per doubling: `docs/paper/figures/README.md` (fig-morphlm). A reproduction (PR #1021) has not landed.



Branch results, not reproducible from main: the study's code and logs are on branch claude/intelligent-shannon-b6n6pj (1755a3f6); nothing was re-run for this figure. Matched data means the same bytes, not the same passes (the Llama arm trained several passes, the organism one). Held-out bpb is frozen (nothing learned while scoring). Values are parsed from docs/research/morph-lm-summary.md §3, which names each column's source; the columns are different organisms. Pretrained open models score 0.82-1.31 on the same held-out text (§3).

Figure 11. Bits per byte against training bytes, a morph trained from scratch against a Llama-style transformer on the same bytes: the morph is lower up to 128 million bytes and ▼ higher from 256 million. Same bytes, not the same passes. One line mixes in a small transformer partner and is not the morph alone. Unreproduced: from an unmerged research branch.